Reflection

What's the worst case for your algorithm, and what kind of input triggers it?

The worst case single run time complexity for this algorithm is $O(n^2 \log n)$ in a max density graph. However, the worst case for Karger's algorithm is $O(mn^2 \log^2 n)$. This is because the time complexity for my implementation that uses edge key, sorting and union find is $O(m \log n)$. The probability bound on a single run given a minimum cut is on a single run is low at $\frac{2}{n(n-1)}$. Thus to reach a 99% success rate, it has to be repeated $O(n^2 \log n)$ times thus the worst case is $O(mn^2 \log^2 n)$.

What was the hardest bug or design decision you encountered, and how did you resolve it?

The hardest bug/bottleneck I encountered was when each contraction step re-scans every edge and relabels every vertex. This bug was a result of my old implementation being kind of brute forced and suboptimal thus resulting in one trial being O(n*m) instead of O(m). To resolve this bug, I implemented the current solution which was to sort the edges using a key. Basically the algorithm assigned every edge a key in which it was sorted in ascending order. I then implemented a union find to use these sorted edges and process them and eliminate self loops until it finds the cut.

If you did this implementation again from scratch, what would you do differently?

If I had to implement this again from scratch, I would implement a single pass union find instead of my current two pass union-find find function that has one pass finding the root and the other compressing the path. I think implementing a single pass union-find find function would optimise the algorithm to run more efficiently than the current two pass one and also clean up the code with fewer traversals.

What's one thing about your algorithm that you didn't fully understand before starting, and now do?

The one thing I definitely did not understand was implementation of the key. The key itself is random based off the edge's weight and is assigned uniquely to every edge. This is through exponential distribution and a negative natural logarithm as seen "long double key = -std::log(u) / e.w;". The key is responsible for the algorithm working as it sorts all the edges in ascending order using the union find. This sorting is what makes this algorithm more efficient and allows it to have the $O(m \log n)$ time complexity.